OPENPA

An Open-Source Personal Assistant-as-a-Service
with LLM-Driven Tool Orchestration
and Autonomous Code Generation

Maxwell Au (U2321963E) · College of Computer and Data Science
Nanyang Technological University · April 2026

THE PROBLEM

> *The modern professional juggles 15+ fragmented services daily—each with its own interface, notifications, and workflow.*

Gmail | GitHub | Spotify | Discord | Telegram | Calendar | Mastodon | RSS | YouTube | Slack | Notion | LinkedIn

Commercial assistants (Siri, Alexa, Google) are:

- Vendor-locked — can't cross ecosystem boundaries
- Shallow — single-step commands only ("set a timer")
- Closed — no self-hosting, no code generation, no customisation

MEANWHILE, DEVELOPERS EVERY MORNING...

Alt+Tab → Gmail → Alt+Tab → GitHub → Alt+Tab → Discord → Alt+Tab → Spotify

"Check my PRs, summarise them, and send the summary to my team on Discord"
→ impossible with any existing assistant

RESEARCH GAP

System	Multi-Svc	Code Gen	Self-Evolve	RAG Mem.	Self-Host	Multi-LLM	Multi-User
Google Assistant	Partial	✗	✗	✗	✗	✗	✗
Siri / Alexa	Partial	✗	✗	✗	✗	✗	✗
Claude Code	✗	✓	✗	✗	✗	✗	✗
Codex CLI	✗	✓	✗	✗	✓	✗	✗
LangChain / AutoGPT	✗*	✗	✗	Partial	✓	✓	✗
OpenPA	✓	✓	✓	✓	✓	✓	✓

*Frameworks, not end-user applications; require custom integration per service.

WHAT IS OPENPA?

An open-source, self-hosted personal assistant that orchestrates
16 services through a single LLM-powered chat interface.

88
MCP Tools

16
Integrated Services

6
LLM Providers

KEY CONTRIBUTIONS

1. UNIFIED MULTI-SERVICE ORCHESTRATION

88 MCP-compliant tools across 16 services, accessible through natural language with per-user credential isolation.

2. RAG-BASED PERSONALISATION

Learns about you over time using locally-hosted embeddings (Ollama) and ChromaDB. Injects relevant memories per conversation.

3. AUTONOMOUS VIBE-CODING

15 workspace tools: clone, edit, grep, test, fix, commit, push, PR — with adaptive test-fix loops.

4. SELF-EVOLVING ARCHITECTURE

The agent can modify its *own* codebase. "Add a LinkedIn tool" → reads patterns, generates code, tests, PRs.

SYSTEM ARCHITECTURE

Frontend

Next.js 16 · React 19 · SSE Streaming · Settings UI

↓ REST + SSE ↓

Backend

FastAPI · Agent Loop · LLM Providers · MCP Tool Registry · RAG

↓ SQLite + ChromaDB ↓

Data Layer

Users · Conversations · Credentials · Vector Embeddings · Semantic Memory

MCP TOOL ARCHITECTURE

Each service = self-contained FastMCP server, mounted into a composite registry with namespaces.

```
mcp = FastMCP("openpa")
mcp.mount(gmail_mcp,     namespace="gmail")      # gmail_send_email, gmail_get_unread, ...
mcp.mount(github_mcp,    namespace="github")     # github_create_pr, github_list_issues, ...
mcp.mount(workspace_mcp, namespace="ws")         # ws_workspace_run, ws_workspace_edit, ...
mcp.mount(spotify_mcp,   namespace="spotify")    # spotify_play, spotify_search, ...
# ... 12 more services
```

GitHub (13) Workspace (15) Memory (8) Gmail (4) Mastodon (9) Spotify (5) Discord (4)

Telegram (4) Calendar (3) Sandbox (7) Scheduler (3) Web Scrape (3) RSS (5) YouTube (2)

Web Search (1) Weather (2)

ADAPTIVE AGENT LOOP

MODE A: PLAN-THEN-EXECUTE

For multi-service orchestration tasks.

```
[
  {step: 1, tool: "web_search"},
  {step: 2, tool: "llm_generate",
   depends_on: 1},
  {step: 3, tool: "telegram_send",
   depends_on: 2}
]
```

Steps with **depends_on** receive outputs from referenced steps.

MODE B: ADAPTIVE LOOP

For coding tasks and iterative workflows.

```
while iterations < max:
  response = llm.chat(
    messages, tools, system
  )
  if response.has_tool_calls:
    result = call_tool(...)
    # LLM sees result next turn
  else:
    break  # done
```

80 iterations for code · 30 for general tasks

LLM HARNESS & GUARDRAILS

Controlling LLM behaviour across 6 providers with varying capabilities.

STRUCTURED SYSTEM PROMPT

1. Identity & scope — anchors role, prevents refusals
2. Parallelisation rules — "call independent tools together"
3. Output format enforcement — templates for briefings, lists
4. Tool preference hierarchy — workspace vs sandbox vs github
5. 20+ few-shot examples — query → expected tool calls

RUNTIME GUARDRAILS

- **Date injection** — LLMs hallucinate dates without it
- **Progress reminders** — every 3 iters, prevent drift
- **Tool-as-text recovery** — detect & nudge on misfire
- **Error-as-result** — LLM reads errors, self-corrects
- **Dynamic tool expansion** — auto-add github/web in coding

Dynamic context sandwich (appended per-turn):

Today's date | User email | Connected services | User preferences | RAG memories (top-10)

CONTEXT-AWARE TOOL FILTERING

88 tools overwhelm small models. OpenPA dynamically filters by task context.

"Add a health check endpoint"	→	workspace	github	core	= 36 tools
"Play some jazz"	→	media	core		= 15 tools
"Who am I?"	→	core			= 8 tools

88 → 8

Enables 7–9B local models (Gemma, Qwen, Llama) to function effectively

RAG-BASED PERSONALISATION

Not just preferences — a semantic understanding that grows over time.

HOW IT WORKS

- **Acquire** — LLM calls `remember_about_user` proactively
- **Embed** — `nomic-embed-text` via Ollama (local, free)
- **Store** — ChromaDB with user ID + category metadata
- **Retrieve** — Top-10 by cosine similarity per message
- **Inject** — Appended to system prompt with relevance scores

EXAMPLE

```
User (day 1): "I'm a software dev
who loves hiking and Italian food."

→ Saves 2 memories (work, interests)

User (day 7): "Who am I?"

→ RAG retrieves:
  [work] Software developer (0.87)
  [interests] Hiking, Italian food (0.82

→ Agent recalls everything
```

MULTI-PROVIDER LLM SUPPORT

Bring Your Own Key — the server stores **no** provider credentials.

ANTHROPIC

Claude Sonnet 4, Haiku

GOOGLE

Gemini 2.5 Flash / Pro

OPENAI

GPT-4o, 4o-mini

GROQ

Llama 3.3 70B (fast)

OLLAMA

Local: Gemma, Qwen, Llama

OPENROUTER

200+ models, single key

Users pay their own bills · No vendor lock-in · Privacy with local models

VIBE-CODING WORKSPACE

The agent autonomously clones, reads, edits, tests, fixes, and ships code.

15 WORKSPACE TOOLS

Explore: list_files, read_file, grep, find, inspect (AST)

Modify: write_file, edit_file, delete_file, install

Verify: run (pytest, npm build), check_syntax

Git: diff, commit_push, create, cleanup

4-PHASE ADAPTIVE WORKFLOW

- **Understand** — Clone, explore, grep patterns
- **Research** — Web search for unfamiliar APIs
- **Implement** — Edit code, check syntax
- **Verify (loop)** — Run tests; if fail → fix → retest

EXAMPLE: VIBE-CODING IN ACTION

User: *"Add a health check endpoint to the REST API."*

- `ws_workspace_create` — Clone repo, branch `feature/health-check`

EXAMPLE: VIBE-CODING IN ACTION

User: *"Add a health check endpoint to the REST API."*

- `ws_workspace_create` — Clone repo, branch `feature/health-check`
- `list_files` + `read_file` + `grep` — Explore codebase, learn conventions

EXAMPLE: VIBE-CODING IN ACTION

User: *"Add a health check endpoint to the REST API."*

- `ws_workspace_create` — Clone repo, branch `feature/health-check`
- `list_files` + `read_file` + `grep` — Explore codebase, learn conventions
- `edit_file` — Add `/api/health` endpoint

EXAMPLE: VIBE-CODING IN ACTION

User: *"Add a health check endpoint to the REST API."*

- `ws_workspace_create` — Clone repo, branch `feature/health-check`
- `list_files` + `read_file` + `grep` — Explore codebase, learn conventions
- `edit_file` — Add `/api/health` endpoint
- `check_syntax` — py_compile + ruff ✓

EXAMPLE: VIBE-CODING IN ACTION

User: *"Add a health check endpoint to the REST API."*

- `ws_workspace_create` — Clone repo, branch `feature/health-check`
- `list_files` + `read_file` + `grep` — Explore codebase, learn conventions
- `edit_file` — Add `/api/health` endpoint
- `check_syntax` — py_compile + ruff ✓
- `run` — `pytest tests/ -v` → 32/32 pass ✓ `npm run build` → ✓

EXAMPLE: VIBE-CODING IN ACTION

User: *"Add a health check endpoint to the REST API."*

- `ws_workspace_create` — Clone repo, branch `feature/health-check`
- `list_files` + `read_file` + `grep` — Explore codebase, learn conventions
- `edit_file` — Add `/api/health` endpoint
- `check_syntax` — py_compile + ruff ✓
- `run` — `pytest tests/ -v` → 32/32 pass ✓ `npm run build` → ✓
- `commit_push` + `github_create_pr` — Ship it!

EXAMPLE: VIBE-CODING IN ACTION

User: *"Add a health check endpoint to the REST API."*

- `ws_workspace_create` — Clone repo, branch `feature/health-check`
- `list_files` + `read_file` + `grep` — Explore codebase, learn conventions
- `edit_file` — Add `/api/health` endpoint
- `check_syntax` — py_compile + ruff ✓
- `run` — `pytest tests/ -v` → 32/32 pass ✓ `npm run build` → ✓
- `commit_push` + `github_create_pr` — Ship it!

Fully autonomous · if tests fail, it reads the error, fixes, and re-runs

THE ASSISTANT THAT EXTENDS ITSELF

OpenPA's source code is on GitHub.
The vibe-coding system can modify its own codebase.

User says:

"Add a LinkedIn tool to OpenPA."

Agent autonomously:

1. Clones itself → 2. Reads existing tool patterns → 3. Generates new tool → 4. Registers in registry → 5. Writes tests → 6. Runs test suite → 7. Opens PR for review

SELF-EVOLUTION SAFETY

Not as scary as it sounds. 🛡️

PULL REQUESTS, NOT MERGES

Changes are submitted as PRs. A human reviews before deployment.

FULL TEST SUITE MUST PASS

Broken code is never shipped. CI validates every push.

ISOLATED WORKSPACE

Agent works on a cloned copy, not the live deployment.

AUTO-CLEANUP

Workspace is deleted after push. No resource leakage.

EXAMPLE: CROSS-SERVICE WORKFLOW

"My friend Alex wants to open a coffee roastery. Research the best specialty coffee shops globally, pick up the naming trend, propose 10 names, and message him on Telegram."

- memory `remember_about_user` — Save "friend Alex wants coffee roastery"

EXAMPLE: CROSS-SERVICE WORKFLOW

"My friend Alex wants to open a coffee roastery. Research the best specialty coffee shops globally, pick up the naming trend, propose 10 names, and message him on Telegram."

- memory `remember_about_user` — Save "friend Alex wants coffee roastery"
- web `web_search` ×2 — Best specialty coffee shops SG + globally

EXAMPLE: CROSS-SERVICE WORKFLOW

"My friend Alex wants to open a coffee roastery. Research the best specialty coffee shops globally, pick up the naming trend, propose 10 names, and message him on Telegram."

- memory `remember_about_user` — Save "friend Alex wants coffee roastery"
- web `web_search` ×2 — Best specialty coffee shops SG + globally
- llm `llm_generate` — Analyse trends, propose 10 original names

EXAMPLE: CROSS-SERVICE WORKFLOW

"My friend Alex wants to open a coffee roastery. Research the best specialty coffee shops globally, pick up the naming trend, propose 10 names, and message him on Telegram."

- memory `remember_about_user` — Save "friend Alex wants coffee roastery"
- web `web_search` ×2 — Best specialty coffee shops SG + globally
- llm `llm_generate` — Analyse trends, propose 10 original names
- telegram `telegram_send_message` — Send to Alex

EXAMPLE: CROSS-SERVICE WORKFLOW

"My friend Alex wants to open a coffee roastery. Research the best specialty coffee shops globally, pick up the naming trend, propose 10 names, and message him on Telegram."

- memory `remember_about_user` — Save "friend Alex wants coffee roastery"
- web `web_search` ×2 — Best specialty coffee shops SG + globally
- llm `llm_generate` — Analyse trends, propose 10 original names
- telegram `telegram_send_message` — Send to Alex

Memory + Web Research + Text Generation + Messaging — one natural language request

"GIVE ME A DAILY BRIEFING"

Step 1 (parallel):

`gmail_get_unread()` → 3 emails
`calendar_list_events()` → 2 meetings
`github_list_notifications()` → 5 PRs
`rss_fetch_all_feeds()` → 12 articles

Step 2:

`llm_generate()`
"Summarise into a concise briefing"

→ One summary from 4 services
→ One turn of conversation

FUTURE: THE RALF LOOP

From evolving *on demand* to evolving *proactively*.

R
Research
Scrape trends, monitor issues

→

A
Assess
Rank by feasibility, demand, value

→

L
Launch
Vibe-code it, test, ship PR

→

F
Feedback
Monitor usage, iterate

A continuously self-improving assistant that discovers, builds, and refines new capabilities autonomously.

LIMITATIONS

- **SQLite** limits horizontal scaling — needs PostgreSQL for production

LIMITATIONS

- **SQLite** limits horizontal scaling — needs PostgreSQL for production
- **Local models** (7–9B) still struggle with complex multi-step tool-calling

LIMITATIONS

- **SQLite** limits horizontal scaling — needs PostgreSQL for production
- **Local models** (7–9B) still struggle with complex multi-step tool-calling
- **Security** — OAuth tokens stored without encryption at rest

LIMITATIONS

- **SQLite** limits horizontal scaling — needs PostgreSQL for production
- **Local models** (7–9B) still struggle with complex multi-step tool-calling
- **Security** — OAuth tokens stored without encryption at rest
- **Test coverage** — core tests exist, but no full end-to-end agent loop tests

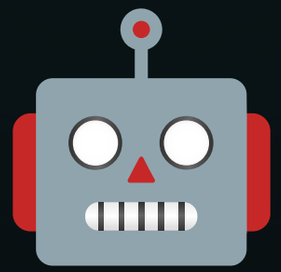
TECH STACK

Python 3.14 | FastAPI | FastMCP | ChromaDB | SQLite | Next.js 16 | React 19

Tailwind CSS | Ollama | SSE Streaming | OAuth 2.0 | JWT Auth | Docker Compose

github.com/maxwellau2/OpenPA

THANK YOU

Questions?

Maxwell Au (U2321963E) · NTU CCDS · April 2026
github.com/maxwellau2/OpenPA